



On-device Visual Perception for Light-weight Browser Agents

PROBLEM STATEMENT ID

SIH26171

THEME

Miscellaneous

CATEGORY

Software

TEAM NAME

ByteForce

TEAM LEADER

Shrijal Gupta

INSTITUTE

IIT Madras BS Degree Programme

Team Details



Member	Name	Role / Contribution
Team Leader	Shrijal Gupta	Python Developer, LangChain, ML, NLP
Member 2	Mohd Afsar	Full-Stack Developer (JavaScript, TypeScript, FastAPI)
Member 3	Tejas Patare	Backend Developer (FastAPI, Django)
Member 4	Sushama Kumari	Computer Vision, NLP, Python, QA
Member 5	Garv Singh	AI/ML, Deployment, QA
Member 6	Ehtesham Aalam	UI/UX Designer, TypeScript, Chrome Manifest v3



Idea / Approach / Proposed Solution

1

The Problem

Next-gen AI agents require visual context (screen state, UI elements) to assist users with complex workflows, but cloud-hosted AI pipelines expose sensitive personal data. Transmitting raw screenshots or DOM trees sends Passwords, Credit Card numbers, Bank details, PII (Personally Identifiable Information), and Faces to third-party cloud servers.

2

Our Solution

A dual-engine browser extension + server architecture that combines local edge-privacy with cloud intelligence. Dynamically detects & redacts sensitive visual elements (blacking out passwords, blurring faces, masking credit cards/SSNs) before any network packet leaves the machine. Transmits only anonymized visual layouts, structural metadata, and redacted images to the server.

3

What Makes It Different

Our privacy-preserving agent combines client-side WebGPU acceleration with cloud reasoning to sanitize DOM text and visual pixels in under 50ms before transmission. This hybrid edge-redaction design ensures that detected sensitive PII is redacted locally before server transmission while keeping the local hardware footprint minimal. As a result, users get the advanced intelligence of cloud-scale models with privacy-preserving processing and low latency.

Novelty and Uniqueness



EXISTING SOLUTIONS

- **Cloud-only VLM Agents (e.g., Multi-On, Operator):** Offer strong task execution but require raw video/screenshot streams, exposing passwords, financial data, and credentials to third-party cloud servers.
- **Rule-based DOM Masking Extensions:** Fast, but rely strictly on static regex or CSS selectors; completely fail on Canvas rendering, complex UI widgets, dynamic frames, and text inside screen images.
- **Fully Local Heavy LLM/VLM Models:** Provide total privacy, but demand immense GPU/RAM, causing severe latency, battery drain, and browser crashes on standard user hardware.

WHAT'S GENUINELY NEW

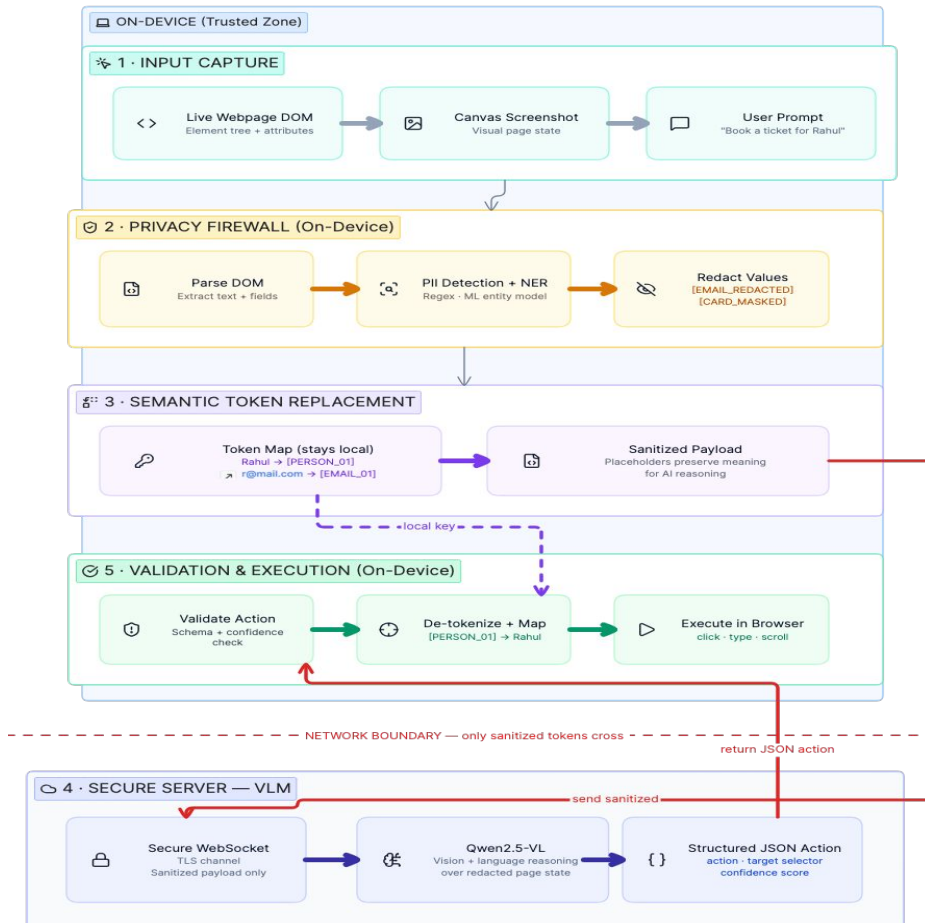
- **Hybrid Dual-Engine Processing:** Combines local WebGPU-accelerated ViT visual detection with DOM-level semantic masking to detect and redact sensitive content locally before server transmission.
- **Sanitized Visual Schema Mapping:** Replaces sensitive visual areas with lightweight, structured placeholder tags that allow cloud VLMs to understand the relevant page state. eg. ([CARD_REDACTED], [EMAIL_MASKED])
- **Privacy-Preserving Low-Latency Processing:** Reduces exposure of sensitive information by keeping detected sensitive data on the client while sending only sanitized visual metadata to the cloud.
- The server doesn't just analyze, it returns executable action commands. The browser agent validates and executes them (click, type, scroll), captures the new state, re-redacts, and loops.

Technical Approach

SUGGESTED: PROCESS FLOW DIAGRAM

Privacy-Preserving Agentic AI Browser Automation

Raw PII never leaves the device — only sanitized, tokenized context crosses the network



Process Flow Diagram (Details)

- **[Input]:** Live Webpage DOM + Canvas Screenshot + User Prompt
- **[Process]:** On-Device WebGPU Engine (4-Layer Redaction & NER) (e.g. [EMAIL_REDACTED], [CARD_MASKED])
- **[Logic]:** Semantic Token Replacement → WebSocket Payload Generation
- **[Output]:** Server VLM (Qwen2.5-VL) returns Structured JSON Action Schema "action": "click", target "button #place-order", "confidence": 0.97
- **[User]:** Local Action Validation → Browser Execution (click, type, scroll)

Technical Approach



TECH STACK

- **Frontend:** TypeScript, Web Browser Extension (Manifest V3), Web APIs
- **Backend:** Python 3.11, FastAPI, Async WebSockets, vLLM Engine
- **Database / Storage:** Supabase (System Config), Chrome storage.local (Client-side Security Policies)
- **AI/ML & Core Tech:** ONNX Runtime Web (WebGPU / WASM SIMD), Qwen2.5-VL-7B-Instruct, Transformers.js, Quantized BlazeFace / YOLOs-tiny
- **Hosting / Deployment:** Render / Vercel (API Gateway), Local On-Device Execution (Browser Extension)

Feasibility and Viability



Potential Challenges

- **Client Resource & Thread Latency:** Running local vision transformers and NER models inside a browser tab can exhaust memory limits (WASM 4GB cap) and freeze the UI thread during real-time page rendering.
- **Over-Redaction vs. PII Leakage Trade-off:** Over-redacting screen regions strips critical structural context needed by the server VLM to reason, while under-redaction risks leaking sensitive user PII.
- **Hardware & Browser Heterogeneity:** WebGPU support varies across client GPUs, operating systems, and browser platforms, leading to potential performance drops on low-end hardware.
- **Dynamic DOM & Grounding Fragility:** Single Page Applications (SPAs), Shadow DOMs, and responsive canvas layers make coordinate-only or raw CSS selector action execution fragile.
- **Security & Prompt Injection Risks:** Untrusted web text containing hidden adversarial prompts can trick the server-side VLM into issuing unauthorized or harmful browser actions.



Mitigation Strategy

- **Offscreen Execution & Model Quantization:** Run INT8 quantized ONNX models (BlazeFace, YOLOs-tiny) inside Manifest V3 Offscreen Documents to isolate thread execution, keeping extension memory under 150MB and local latency under 50ms.
- **Semantic Preserving Tokenization:** Replace sensitive text/values with structured labels (e.g., [REDACTED_EMAIL], [REDACTED_NAME]) and tight bounding box overlays instead of full-screen blackouts, preserving layout utility for the VLM.
- **Adaptive Hardware Acceleration Architecture:** Implement automatic fallback triggers: if WebGPU acceleration is unavailable or fails, dynamically downgrade execution to multi-threaded WASM SIMD.
- **Dual-Spectrum Hybrid Grounding:** Combine pruned HTML DOM attributes, Accessibility (A11y) tree node IDs, and spatial coordinates to accurately resolve target elements across dynamic web interfaces.
- **Zero-Trust Policy Engine & Sanitization:** Pre-sanitize incoming web page content before VLM ingestion and route all server-generated actions through a client-side policy validator that requires user confirmation for high-risk operations.

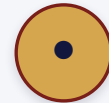


Impact and Benefits



Social Impact

Builds trust in AI-assisted digital services by keeping sensitive information such as credentials, financial details and personal data on the user's device wherever possible. Supports safer and more privacy-conscious adoption of AI.



Economic Impact

Reduces the need to transmit and centrally process raw sensitive screen data, potentially lowering data-exposure risk, security overhead and compliance burden while enabling scalable AI-assisted workflows.



Environmental Impact

Lightweight on-device inference can reduce unnecessary data transmission and some remote processing workloads, potentially lowering network and server-side resource usage.



Research and References

REFERENCES

- **Google Research:** *BlazeFace: Fast On-Device Face Detection*, 2019, arxiv.org/abs/1907.05047
- **Meta AI / Hugging Face:** *YOLOS: You Only Look at One Sequence for Object Detection*, 2021, arxiv.org/abs/2106.00666
- **W3C WebGPU Working Group:** *WebGPU Architecture Specification for Client-Side ML Acceleration*, 2023, w3.org/TR/webgpu
- **Chrome Extension API Team:** *Manifest V3 Specifications & Offscreen Documents Guide*, 2023, developer.chrome.com/docs/extensions
- **ByteDance AI Lab / UI-TARS:** *Vision-Language Models for Autonomous Web GUI Grounding and Action Execution*, 2024, arxiv.org/abs/2501.12326
- **MeitY, Government of India (2023)** — *Digital Personal Data Protection Act, 2023* [MeitY – DPDP Act](#)

Solution Summary



Criterion	Summary
Problem	<i>Cloud-based visual browser agents may expose sensitive screen information such as names, emails, passwords and other PII to external servers. Fully on-device agents improve privacy but are constrained by browser-side compute and memory resources.</i>
Proposed Solution	<i>Our browser agent performs visual perception and sensitive-data detection locally using lightweight models. PII is redacted before any network request, and only sanitized screen context is sent to a server-side VLM for reasoning. The returned action is then executed locally by the browser agent.</i>
Novelty / USP	<i>A hybrid perception approach combines DOM/accessibility information with selective visual processing, while using multi-layer local PII detection and semantic redaction. This preserves useful UI context for AI reasoning without exposing the original sensitive information.</i>
Feasibility	<i>The solution can be implemented as a Chrome/Firefox extension using standard browser technologies, WebGPU and ONNX Runtime Web. Lightweight or quantized models handle local perception, while heavier VLM reasoning remains on the server, balancing accuracy, latency and resource usage.</i>
Impact	<i>The solution enables privacy-aware browser automation while reducing transmission of raw screen data and unnecessary remote processing. It can make AI-assisted web workflows safer and more practical for users and organizations handling sensitive digital information.</i>